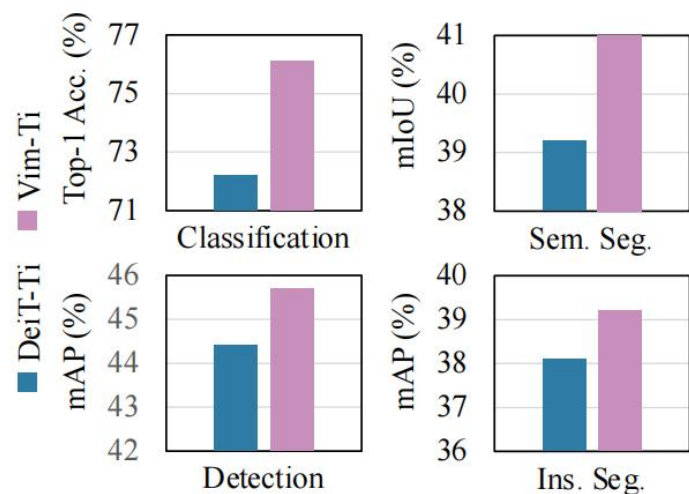


Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model

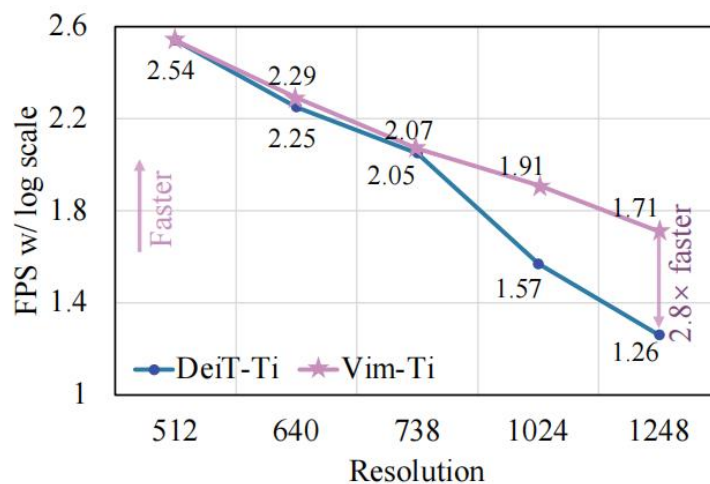
Lianghui Zhu Bencheng Liao
Qian Zhang Xinlong Wang
Wenyu Liu Xinggang Wang

ICML 2024

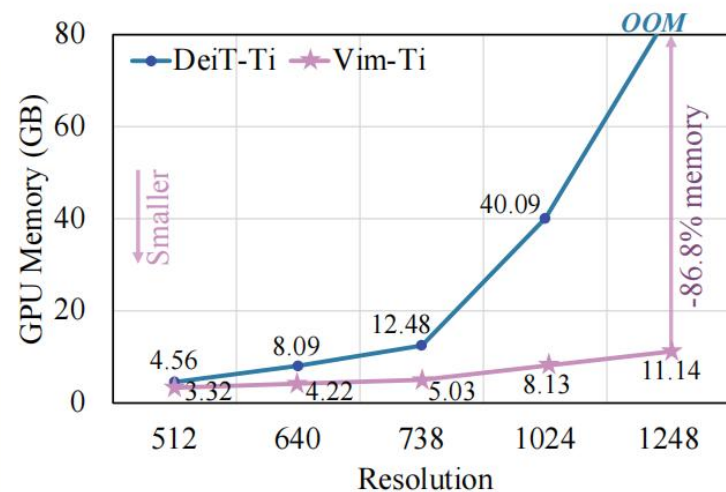
与传统ViT模型的对比



(a) Accuracy Comparison



(b) Speed Comparison



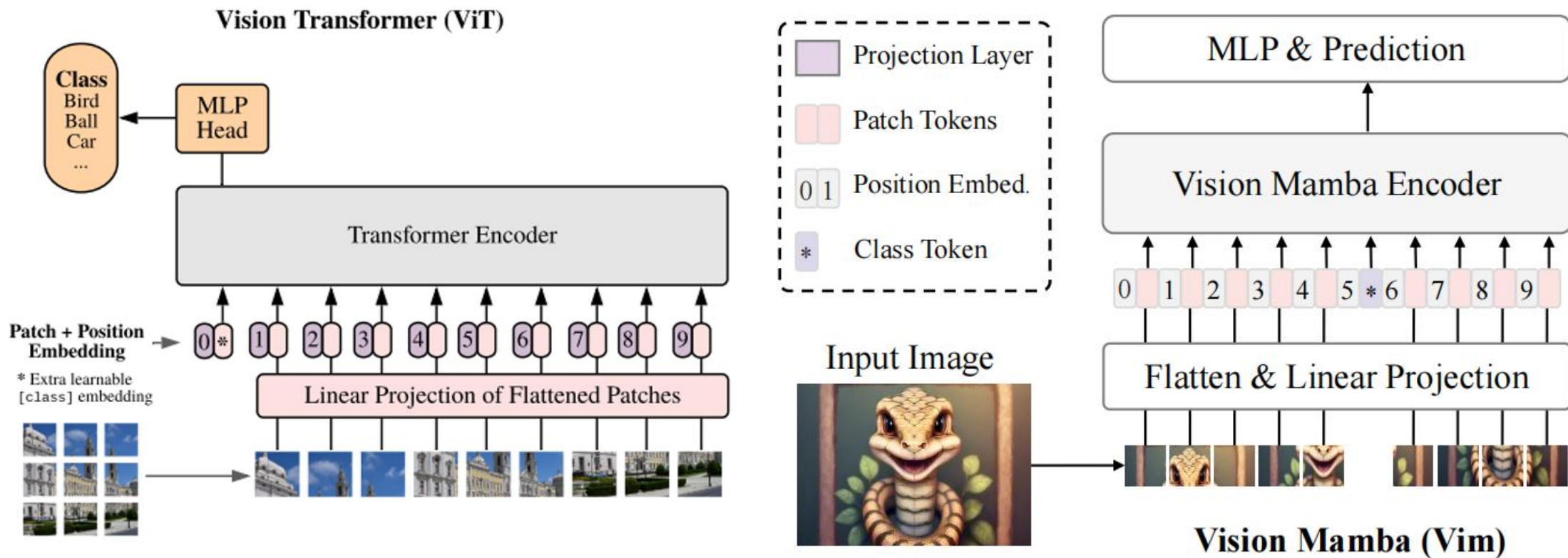
(c) GPU Memory Comparison

Figure 1: Performance and efficiency comparisons between DeiT (Touvron et al., 2021a) and our Vim model. Results show that Vim outperforms DeiT on both ImageNet classification and downstream detection and segmentation tasks and is more computation and memory efficient than DeiT in dealing with high-resolution images. For example, Vim is 2.8 $\times$ faster than DeiT and saves 86.8% GPU memory when performing batch inference to extract features on images with a resolution of 1248×1248 , *i.e.*, 6084 tokens per image.

研究背景

- 解决的核心问题：人工智能在图像识别任务中主要依赖于Vision Transformer (ViT) 模型，但其存在显著的性能瓶颈：计算效率低下和内存占用高，特别是在处理高清大图时，常常导致GPU显存溢出。
- 问题根源分析：ViT核心机制——自注意力机制，此机制要求图像中的每个小块(patch)都必须与其他所有小块进行交互，从而导致计算复杂度以平方级别增长，极大地限制了ViT在实际应用尤其是高分辨率图像处理场景下的效能

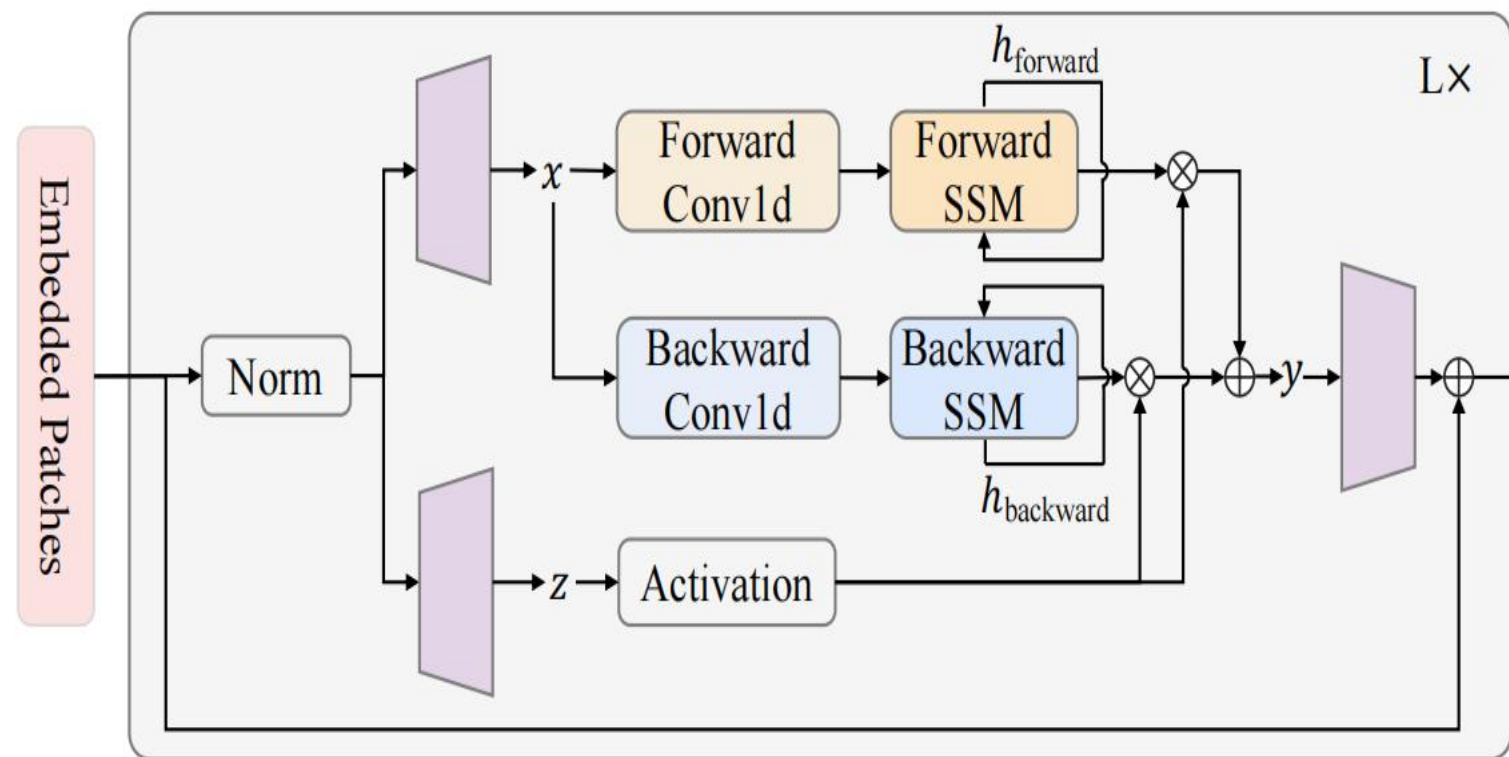
模型架构



Vision Mamba (ViM) 在结构上借鉴了 Vision Transformer (ViT) 的‘图像分块+类标记’范式，但将核心编码器替换为高效的 Mamba 模块

分块 → 投影 → 加位置编码 → 编码器 → 分类

Encoder



Forward SSM (前向状态空间模型)
它会按顺序处理数据, 维护一个隐状态 $h_{forward}$, 只关注“前面的信息”

Backward SSM (后向状态空间模型)
从右到左处理数据, 维护隐状态 $h_{backward}$, 关注“后面的信息”

两个 SSM 对称但方向相反, 实现双向建模

Vision Mamba Encoder

SSM $\longleftrightarrow$ 多头注意力

捕获长距离依赖关系

选择性状态空间模型 (Selective SSM)

- 每一步：读取当前输入；更新内部隐藏状态；输出一个与上下文相关的表示

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (\text{状态方程})$$

$$y_t = Ch_t + Dx_t \quad (\text{输出方程})$$

通过隐状态 (hidden state)
递归地压缩和传递信息

- 选择性

$$B = B(x), \quad C = C(x)$$

根据当前输入内容，动态决定：哪些信息值得写入隐藏状态，
哪些信息值得从状态中读出

Efficiency Analysis

- IO-Efficiency: HBM(大存储)和SRAM(高带宽).预加载关键数据至SRAM, 在SRAM内进行高效计算, 输出结果写回HBM, 让原本需要 $O(BMEN)$ 级别的内存I/O量被成功缩减至 $O(BME + EN)$ 。
- Memory-Efficiency: Vim采用了与Mamba相同的重计算方法, 对于形状为 (B, M, E, N) 的中间状态, Vim会在网络反向传播时对其进行重计算。
- Computation-Efficiency: Vim中的SSM与Transformer中的自注意力机制均在自适应地提供全局上下文方面发挥着关键作用。给定一个视觉序列 $T \in 1 \times M \times D$, 且默认设置 $E=2D$, 则全局自注意力和SSM的计算复杂度分别为:

$$\Omega(\text{self-attention}) = 4MD^2 + 2M^2D,$$

$$\Omega(\text{SSM}) = 3M(2D)N + M(2D)N,$$

算法

Algorithm 1 Vim Block Process

Require: token sequence $\mathbf{T}_{l-1} : (\mathbf{B}, \mathbf{M}, \mathbf{D})$

Ensure: token sequence $\mathbf{T}_l : (\mathbf{B}, \mathbf{M}, \mathbf{D})$

```

1: /* normalize the input sequence  $\mathbf{T}'_{l-1}$  */
2:  $\mathbf{T}'_{l-1} : (\mathbf{B}, \mathbf{M}, \mathbf{D}) \leftarrow \text{Norm}(\mathbf{T}_{l-1})$ 
3:  $\mathbf{x} : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \text{Linear}^x(\mathbf{T}'_{l-1})$ 
4:  $\mathbf{z} : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \text{Linear}^z(\mathbf{T}'_{l-1})$ 
5: /* process with different direction */
6: for  $o$  in {forward, backward} do
7:    $\mathbf{x}'_o : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \text{SiLU}(\text{Conv1d}_o(\mathbf{x}))$ 
8:    $\mathbf{B}_o : (\mathbf{B}, \mathbf{M}, \mathbf{N}) \leftarrow \text{Linear}^B_o(\mathbf{x}'_o)$ 
9:    $\mathbf{C}_o : (\mathbf{B}, \mathbf{M}, \mathbf{N}) \leftarrow \text{Linear}^C_o(\mathbf{x}'_o)$ 
10:  /* softplus ensures positive  $\Delta_o$  */
11:   $\Delta_o : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \log(1 + \exp(\text{Linear}^\Delta_o(\mathbf{x}'_o) + \text{Parameter}^\Delta_o))$ 
12:  /* shape of  $\text{Parameter}^\Delta_o$  is  $(\mathbf{E}, \mathbf{N})$  */
13:   $\overline{\mathbf{A}}_o : (\mathbf{B}, \mathbf{M}, \mathbf{E}, \mathbf{N}) \leftarrow \Delta_o \otimes \text{Parameter}^\Delta_o$ 
14:   $\overline{\mathbf{B}}_o : (\mathbf{B}, \mathbf{M}, \mathbf{E}, \mathbf{N}) \leftarrow \Delta_o \otimes \mathbf{B}_o$ 
15:  /* initialize  $h_o$  and  $y_o$  with 0 */
16:   $h_o : (\mathbf{B}, \mathbf{E}, \mathbf{N}) \leftarrow \text{zeros}(\mathbf{B}, \mathbf{E}, \mathbf{N})$ 
17:   $y_o : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \text{zeros}(\mathbf{B}, \mathbf{M}, \mathbf{E})$ 
18:  /* SSM recurrent */
19:  for  $i$  in {0, ..., M-1} do
20:     $h_o = \overline{\mathbf{A}}_o[:, i, :, :] \odot h_o + \overline{\mathbf{B}}_o[:, i, :, :] \odot \mathbf{x}'_o[:, i, :, \text{None}]$ 
21:     $y_o[:, i, :] = h_o \otimes \mathbf{C}_o[:, i, :]$ 
22:  end for
23: end for
24: /* get gated  $y$  */
25:  $\mathbf{y}'_{\text{forward}} : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \mathbf{y}_{\text{forward}} \odot \text{SiLU}(\mathbf{z})$ 
26:  $\mathbf{y}'_{\text{backward}} : (\mathbf{B}, \mathbf{M}, \mathbf{E}) \leftarrow \mathbf{y}_{\text{backward}} \odot \text{SiLU}(\mathbf{z})$ 
27: /* residual connection */
28:  $\mathbf{T}_l : (\mathbf{B}, \mathbf{M}, \mathbf{D}) \leftarrow \text{Linear}^T(\mathbf{y}'_{\text{forward}} + \mathbf{y}'_{\text{backward}}) + \mathbf{T}_{l-1}$ 
29: Return:  $\mathbf{T}_l$ 

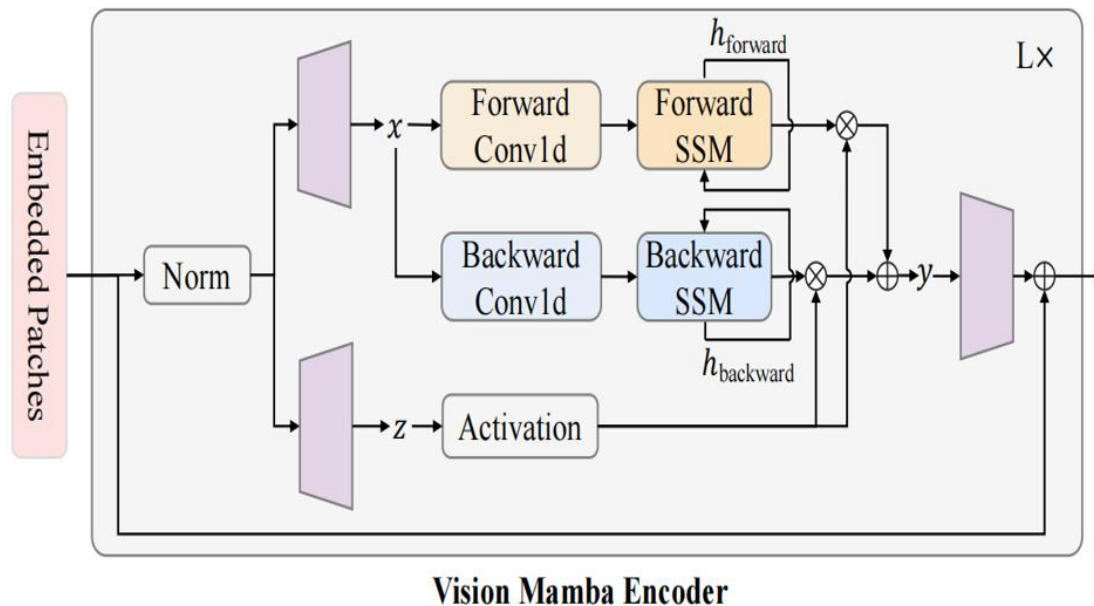
```

- 接收输入 token 序列 $\mathbf{T}_{l-1}$
- 经过归一化、投影、双向 SSM 处理
- 输出增强后的表示 $\mathbf{T}_l$

B: batch size

M: 序列长度 (patch 数)

D: 嵌入维度



实验

Method	image size	#param.	ImageNet top-1 acc.
Convnets			
ResNet-18	224 ²	12M	69.8
ResNet-50	224 ²	25M	76.2
ResNet-101	224 ²	45M	77.4
ResNet-152	224 ²	60M	78.3
ResNeXt50-32×4d	224 ²	25M	77.6
RegNetY-4GF	224 ²	21M	80.0
Transformers			
ViT-B/16	384 ²	86M	77.9
ViT-L/16	384 ²	307M	76.5
DeiT-Ti	224 ²	6M	72.2
DeiT-S	224 ²	22M	79.8
DeiT-B	224 ²	86M	81.8
SSMs			
S4ND-ViT-B	224 ²	89M	80.4
Vim-Ti	224 ²	7M	76.1
Vim-Ti [†]	224 ²	7M	78.3 +2.2
Vim-S	224 ²	26M	80.3
Vim-S [†]	224 ²	26M	81.4 +1.1
Vim-B	224 ²	98M	81.9
Vim-B [†]	224 ²	98M	83.2 +1.3

Table 1: Comparison with different backbones on ImageNet-1K validation set. [†] represents the model is fine-tuned with our long sequence setting.

Method	Backbone	image size	#param.	val mIoU
DeepLab v3+	ResNet-101	512 ²	63M	44.1
UperNet	ResNet-50	512 ²	67M	41.2
UperNet	ResNet-101	512 ²	86M	44.9
UperNet	DeiT-Ti	512 ²	11M	39.2
UperNet	DeiT-S	512 ²	43M	44.0
UperNet	Vim-Ti	512 ²	13M	41.0
UperNet	Vim-S	512 ²	46M	44.9

Table 2: Results of semantic segmentation on the ADE20K val set.

Backbone	AP ^{box}	AP ₅₀ ^{box}	AP ₇₅ ^{box}	AP _s ^{box}	AP _m ^{box}	AP _l ^{box}
DeiT-Ti	44.4	63.0	47.8	26.1	47.4	61.8
Vim-Ti	45.7	63.9	49.6	26.1	49.0	63.2
Backbone	AP ^{mask}	AP ₅₀ ^{mask}	AP ₇₅ ^{mask}	AP _s ^{mask}	AP _m ^{mask}	AP _l ^{mask}
DeiT-Ti	38.1	59.9	40.5	18.1	40.5	58.4
Vim-Ti	39.2	60.9	41.7	18.2	41.8	60.2

Table 3: Results of object detection and instance segmentation on the COCO val set using Cascade Mask R-CNN (Cai & Vasconcelos, 2019) framework.

Bidirectional strategy	ImageNet top-1 acc.	ADE20K mIoU
None	73.2	32.3
Bidirectional Layer	70.9	33.6
Bidirectional SSM	72.8	33.2
Bidirectional SSM + Conv1d	73.9	35.9

Table 4: Ablation study on the bidirectional design. To ensure a fair comparison, we do not use the class token for each experiment. The default setting for Vim is marked in blue .

Classification strategy	ImageNet top-1 acc.
Mean pool	73.9
Max pool	73.4
Head class token	75.2
Double class token	74.3
Middle class token	76.1

Table 5: Ablation study on the classification design. The default setting for Vim is marked in blue .

总结

- Vision Mamba (Vim) 首次将 Mamba 的选择性状态空间模型 (SSM) 成功应用于视觉任务，在保持线性计算复杂度的同时，实现了与 Vision Transformer (ViT) 相当甚至更优的性能。
- Vision Mamba 证明了状态空间模型可以成为 Transformer 在视觉领域的高效替代方案——它以线性复杂度实现了强大的长程建模能力，在精度、速度和可扩展性上取得优异平衡，为下一代视觉基础模型提供了新范式。



感谢观看